UE5 Spline Chase Shooter Project Principles

Purpose

This document stores the default working principles for the current Unreal Engine 5 project.

Base Working Principles

- Act as a coding agent that inspects the codebase before changing it.
- Prefer completing the work end-to-end, including implementation and verification, whenever practical.
- Communicate briefly, clearly, and in a collaborative tone.
- Do not revert user changes unless explicitly asked.
- Avoid destructive Git commands unless the user clearly requests them.
- Share short progress updates while working and keep final summaries focused on outcomes.
- When reviewing code, prioritize bugs, regressions, risk, and missing tests.

Project-Specific Unreal Rules

- Project type: spline-based chase shooter.
- Prefer `Has-a` composition over `Is-a` inheritance.
- Split gameplay features into focused `UActorComponent` units when possible.
- Do not arbitrarily change existing `AActor` or `APawn` inheritance structure.
- Keep components independent.
- Prefer interfaces and delegates over direct component-to-component access.
- Follow Unreal Engine coding conventions.
- Use `PascalCase` naming for variables and reflected members in this project.
- Add `UPROPERTY` declarations with clear categories.

Practical Decision Guide

1. Check whether the requested feature can live in a new or existing component.
2. Reuse the current owner actor or pawn instead of introducing a new inheritance layer.
3. Expose only the data and hooks that other systems actually need.
4. Keep Blueprint integration intentional and readable.
5. Validate that the result still fits spline-driven chase shooter gameplay.

Recommended Component Boundaries

- Spline following
- Pursuit state management
- Target selection

- Shooting or weapon firing
- Hit or damage response
- Encounter or spawn coordination

Output Use

This file is the source text for the companion PDF stored with the skill.